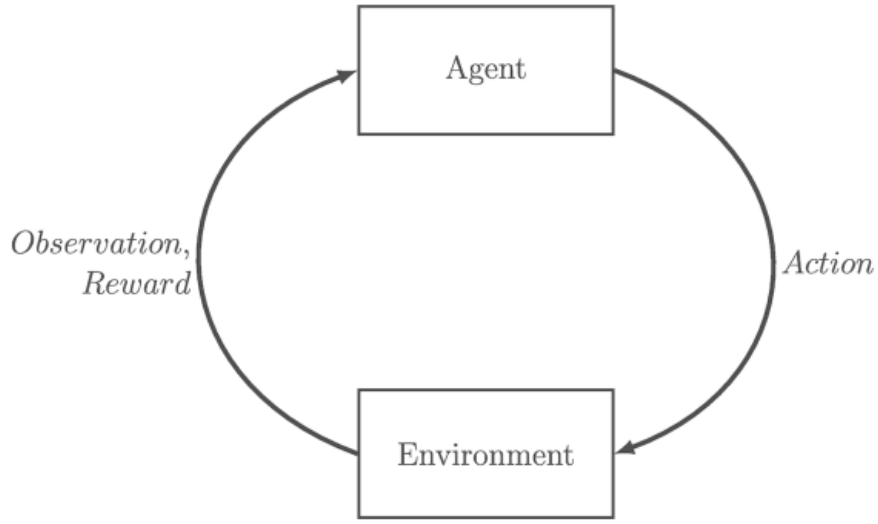
Chapter 2 Markov Decission Proccess (MDP)

29 April 2024 9:59

- MDPs are a powerful framework for sequential decision-making in reinforcement learning across fields like robotics and finance.
- The chapter covers key MDP components, policy and value functions, Bellman equations, and their application in updating values.
- Focus is on finite MDPs with deterministic settings, with a nod to stochastic considerations.
- Highlighted as a crucial chapter for understanding and applying MDPs in reinforcement learning.

1. Overview of MDP
   Markov Decision Processes (MDPs) is a mathematical framework for modeling sequential decision-making problems under uncertainty. It emphasizes representing problems with states, actions, a transition model, and a reward function to derive an optimal policy maximizing expected rewards over time.
   a. Components of an MDP:
      i. States(S): Defined as all possible environment configurations or observations where the agent can exist. Examples include chess board configurations, stock prices in finance, robot positions and velocities, vehicle locations, and inventory levels in supply chains.
      ii. Actions (A): Encompasses all feasible moves the agent can make, such as moving pieces in chess, buying/selling stocks in finance, robot accelerations, vehicle turns, or inventory restocking.
      iii. Transition Models(P): Described as a function determining the probability of transitioning to a new state based on the current state and action taken. It models how the environment reacts to the agent's actions, influenced by factors like game rules, stock price fluctuations, robot dynamics, vehicle movements, or supply chain dynamics.
      iv. Reward functions (R): Specifies the rewards or costs associated with taking actions in specific states, reflecting the agent's task objectives. Examples include assigning values to winning/losing in chess, maximizing profit or minimizing risk in finance, optimizing energy efficiency in robot motion, fuel efficiency in vehicle movement, or profit margins in supply chain operations.
   b. Why MDP is powerful
      i. Structured Framework: MDPs provide a well-defined framework with components like states, actions, transition models, and reward functions, offering a systematic way to model decision-making problems.
      ii. Optimal Policy Derivation: MDPs enable the derivation of optimal policies that maximize the expected sum of rewards over time, guiding agents on the best actions to take in different states.
      iii. Mathematical Analysis: By formulating problems as MDPs, mathematical concepts can be applied to analyze and solve complex decision-making scenarios, making it easier to understand and optimize processes.
      iv. Versatility: MDPs can be customized to fit various domains and applications, from robotics and finance to supply chain management, providing a flexible and adaptable framework for problem-solving.
      v. Powerful Problem-Solving Tool: The structured nature of MDPs allows for efficient modeling and solution of decision-making problems, making them a powerful tool for optimizing processes and maximizing rewards over time.

2. Model Reinforcement Learning Problem using MDP



a. Markov notation, symbols and interaction

   i. MDP Framework Overview:
      1) Definition: A Markov Decision Process (MDP) models a sequence of states, actions, and rewards, essential for studying reinforcement learning.
      2) Example: In a scenario like a robot navigating a maze, states (S_t) represent the robot's location, actions (A_t) are movement choices, and rewards (R_t) indicate progress towards the goal.
   ii. Time Step Indexing:
      1) Symbol Usage: t (t) indexes different stages, representing discrete time steps in the process. t = 0, 1, 2, 3, 4, 5..
      2) Clarification: Time steps are not regular intervals but signify different process stages.
   iii. Reward Notation:
      1) Mathematical Representation: Reward at time step "t" is denoted as R_t, simplifying notation for clarity. $R_t=(S_t, A_t)$ or $R_{t+1} = (S_t, A_t, S_{t+1})$
      2) Alternative Notation: Some use R_{t+1} to emphasize reward dependence on the successor state.
   iv. Agent Interaction: Observation and Action: The agent observes state S_t and takes action A_t in the environment.
      1) Transition and Reward: Environment transitions to a new state and generates reward R_{t+1} conditioned on S_t and A_t.
      2) Feedback Loop: Agent receives reward and successor state, continuing the interaction loop until reaching a terminal state.
   v. Symbol Usage:
      1) Random Variables: State (S), Action (A), and Reward (R) are denoted in uppercase due to their variable outcomes.
      2) Specific Outcomes: Lowercase letters represent specific outcomes of these random variables.
   vi. Foundation for Problem Solving
      1) :Components Interaction: State space, action space, transition model, and reward function collectively describe the environment and agent interactions.
      2) Problem Solving Basis: Understanding the interaction of these elements forms the basis for solving reinforcement learning problems.

b. Markov Property
   i. Definition:
      1) The Markov property is a crucial assumption in Markov Decision Processes (MDPs) for the framework to be applicable.
      2) It states that the future state of the system is independent of the past given the present state.
   ii. Mathematical Formulation:

1) The Markov property can be expressed as: $P[S_{t+1}|S_t,At,S_{t-1},A_{t-1},...]=P[S_{t+1}|S_t,A_t]$
2) This equation signifies that the probability of transitioning to the next state St+1depends only on the current state Stand action At, not on any previous states or actions.
3) The Markov property ensures that the current state contains sufficient information to predict the next state without needing the complete history of past states and actions.
4) It simplifies the modeling process by focusing on the immediate state-action pair for decision-making.

iii. Application Example:
1) Consider a robot navigating a room: If the robot's movement depends only on its current position and action taken, satisfying the Markov property, the MDP framework is applicable.
2) Violation of the Markov property occurs if the robot's movement relies on its entire history, making it challenging to model the environment accurately.

iv. Significance:
1) The Markov property is fundamental in MDPs as it ensures a concise representation of state transitions and simplifies decision-making processes.
2) By adhering to the Markov property, MDPs become more manageable and effective for solving sequential decision-making problems.

c. Summary of Service Dog Example
i. Scenario Description:
1) The scenario involves training a service dog to retrieve an object in a house with three rooms.
2) The dog's task is to find the object in one of the rooms and bring it to its owner or trainer.
3) The task is episodic, ending once the dog successfully retrieves the object.

ii. Environment Setup:
1) The house has three rooms, one containing the target object for the dog to retrieve.
2) The starting state is initialized randomly to introduce variability in the training process.
3) One room leads to the front yard, providing a recreational area for the dog.

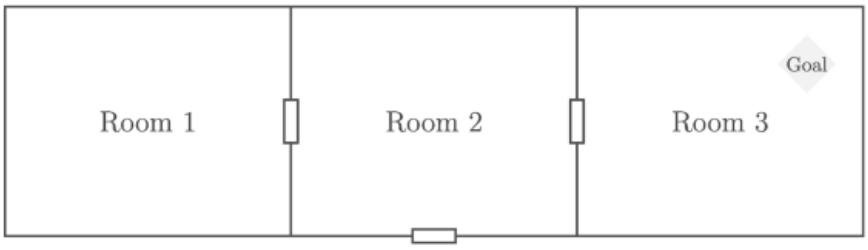
iii. Reinforcement Learning Problem:
1) The task aligns with episodic reinforcement learning, where the goal is achieved upon retrieving the object.
2) The objective is to model this problem as a Markov Decision Process (MDP) for effective decision-making.
3) Modeling and Solution Approach:
a) Modeling: The scenario is structured as an MDP to define states, actions, and rewards for the dog's training task.
b) Dynamics Function: The dynamics of the environment, including state transitions and rewards, are defined to guide the dog's behavior.
c) Policy Construction: A policy is developed to instruct the dog on which actions to take in different states to maximize rewards.
4) Algorithm Implementation:
a) Algorithm Selection: Specific algorithms like dynamic programming, Monte Carlo methods, and temporal difference methods are introduced to solve the service dog reinforcement learning problem.
b) Training Process: These algorithms aid in training the service dog to learn the optimal policy for retrieving the object efficiently.
5) Outcome:
a) By utilizing the service dog example, readers can understand how to apply reinforcement learning concepts in practical scenarios.
b) The example serves as a hands-on illustration of MDPs and reinforcement learning algorithms in action, showcasing their effectiveness in training tasks.

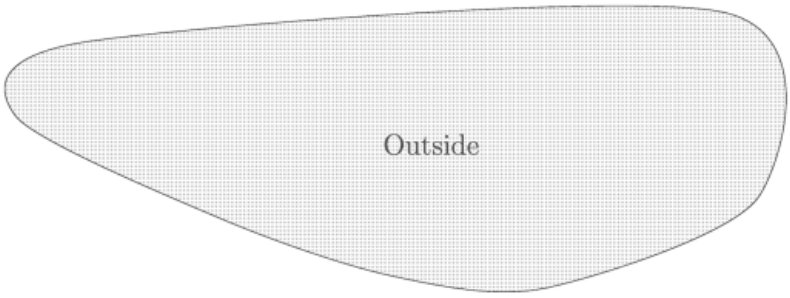
3. Markov Proccess or Markov Chain
a. Markov Process Definition:
i. Markov Process (Markov Chain): A memoryless random process where transitions to a new state depend only on the current state, not on past states or actions.
ii. Symbol: Markov chain represented as ⟨S,P⟩, where Sis the state space and the transition model.

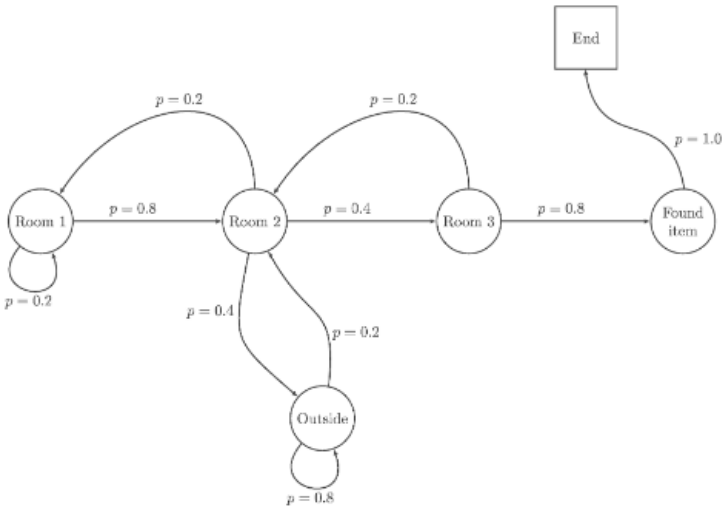
b. Transition Probability:
Formula: Transition probability from state to state s'denoted as P(s,s').
Representation: Graphical depiction with nodes as states and edges showing transitions with associated probabilities.

c. Illustrative Example:

i. Figure: Visual representation of the service dog example as a Markov chain.
ii. Nodes: Represent states (e.g., rooms in the house).
iii. Edges: Show transitions between states with transition probabilities.
iv. Terminal State: Indicated by a square box (e.g., state where the object is found).







$$\mathcal{P} = \begin{array}{c} \\ \text{Room 1} \\ \text{Room 2} \\ \text{Room 3} \\ \text{Outside} \\ \text{Found item} \\ \text{End} \end{array} \begin{array}{c} \begin{array}{cccccc} \text{Room 1} & \text{Room 2} & \text{Room 3} & \text{Outside} & \text{Found item} & \text{End} \end{array} \\ \begin{pmatrix} 0.2 & 0.8 & 0 & 0 & 0 & 0 \\ 0.2 & 0 & 0.4 & 0.4 & 0 & 0 \\ 0 & 0.2 & 0 & 0 & 0.8 & 0 \\ 0 & 0.2 & 0 & 0.8 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1.0 \\ 0 & 0 & 0 & 0 & 0 & 1.0 \end{pmatrix} \end{array}$$

d. Significance of Markov Chains:
   i. Insights: Provide fundamental understanding of state transitions without rewards or actions.
   ii. Influence Analysis: Study how states in a sequence influence each other, aiding in decision-making processes.
e. Application in MDPs:
   i. Foundation: Markov chains serve as a basis for understanding more complex Markov Decision Processes (MDPs).
   ii. Simplification: Focus on state transitions in isolation, laying the groundwork for MDP modeling.
f. Mathematical Formulation:

i. Markov Property: Future state independence on past given present: $P[S_{t+1}|S_t,A_t,S_{t-1},A_{t-1},...]=P[S_{t+1}|S_t,A_t]$
ii. Memoryless Property: Transition probabilities determined solely by current state and action.

g. Educational Value:
   i. Foundational Knowledge: Understanding Markov chains essential for grasping more advanced MDP concepts.
   ii. Insight Development: Analysis of state transitions aids in decision-making strategies in sequential environments.

h. Application Scope:
   Markov chains applicable in various fields beyond reinforcement learning, including stochastic processes and system modelling.

i. Episode Example
   Episode 1: (Room 1, Room 2, Room 3, Found item, End)
   Episode 2: (Room 3, Found item, End)
   Episode 3: (Room 2, Outside, Room 2, Room 3, Found item, End)
   Episode 4: (Outside, Outside, Outside, ...)

4. Markov Reward Process

Definition: Markov Reward Process (MRP): An extension of Markov chains where rewards are incorporated into the state transitions.

Representation: MRP denoted as $\langle S,P,R,\gamma \rangle$, where:

- S: Finite state space.
- P: Transition model specifying probabilities of state transitions.
- R: Reward function providing rewards for state transitions.
- γ: Discount factor for future rewards.

Reward Incorporation:

i. **Objective**: Integrate rewards into the Markov chain to guide the agent's behavior towards maximizing cumulative rewards.
ii. **Reward Signal**: Provided along with state transitions to influence the agent's decision-making process.

Formula Explanation:

i. **Transition Probability**: $P(s,s')$represents the probability of transitioning from state sto state s'.
ii. Reward Function: $R(s)$assigns a reward to each state sbased on the environment's feedback.
iii. Discount Factor: γscales future rewards to emphasize immediate rewards or long-term planning.
iv. Expected Return: The expected sum of discounted rewards starting from state sis calculated as: $G_t=R_{t+1}+\gamma R_{t+2}+\gamma 2R_{t+3}+...=\sum k=0\text{-}\infty \gamma k R_{t+k+1}$

Agent Learning:

i. **Objective**: Agent aims to learn a policy that maximizes the expected return over time.
ii. Value Function: Represents the expected return from a given state under a specific policy.
iii. Bellman Equation: Used to iteratively update the value function based on the rewards and transitions.

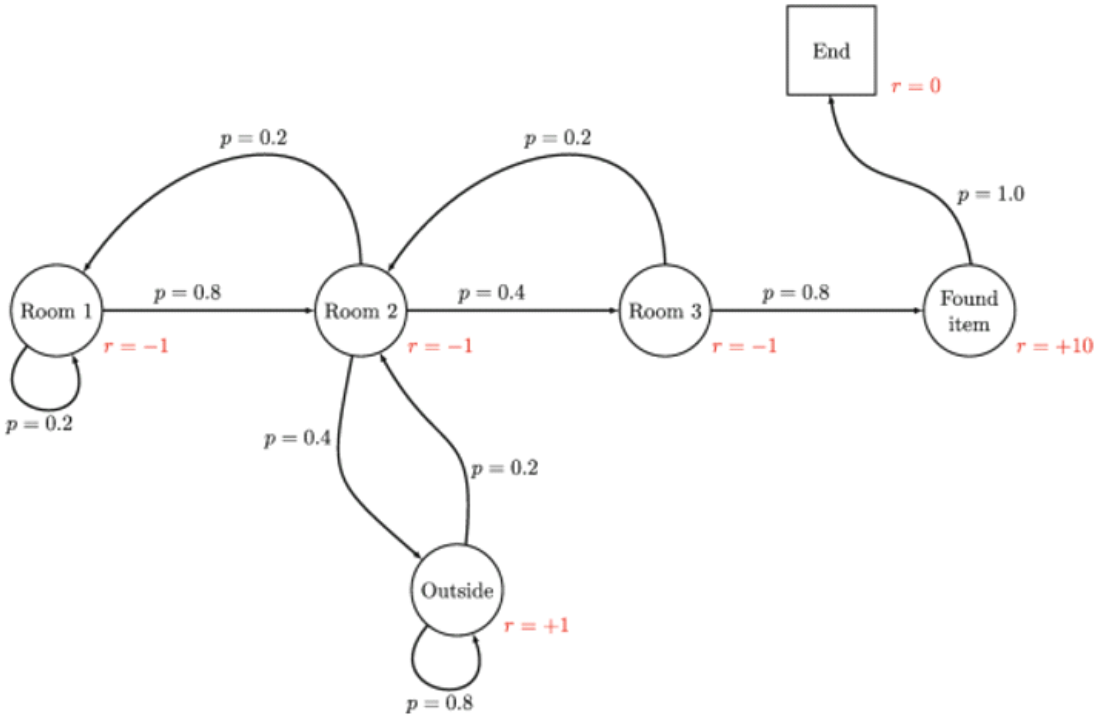
Temporal Aspects:

i. **Temporal Difference Learning**: Updates value estimates based on the difference between current and predicted future rewards.
ii. Exploration-Exploitation Trade-off: Balancing between exploring new states and exploiting known high-reward states.

Application:

i. **Reinforcement Learning**: MRP serves as a fundamental concept in reinforcement learning for modeling reward-driven decision-making processes.
ii. Algorithm Implementation: Various algorithms like Monte Carlo methods and temporal difference learning are applied to solve MRP problems efficiently.

Example Service Dog Markov Reward Proccess



- Episode 1: (Room 1, Room 2, Room 3, Found item, End)
- **Total rewards** $= -1 - 1 - 1 + 10 + 0 = 7.0$
- Episode 2: (Room 3, Found item, End)
- **Total rewards** $= -1 + 10 = 9.0$
- Episode 3: (Room 2, Outside, Room 2, Room 3, Found item, End)

- **Total rewards** $= -1 + 1 - 1 - 1 + 10 + 0 = 8.0$
- Episode 4: (Outside, Outside, Outside ...)
- **Total rewards** $= 1 + 1 + \cdots = \infty$

factor $\gamma = 0.9$:

- Episode 1: (Room 1, Room 2, Room 3, Found item, End)
- $G_0 = -1 - 1 * 0.9 - 1 * 0.9^2 + 10 * 0.9^3 = 4.6$
- Episode 2: (Room 3, Found item, End)
- $G_0 = -1 + 10 * 0.9 = 8.0$
- Episode 3: (Room 2, Outside, Room 2, Room 3, Found item, End)
- $G_0 = -1 + 1 * 0.9 - 1 * 0.9^2 - 1 * 0.9^3 + 10 * 0.9^4 = 4.9$
- Episode 4: (Outside, Outside, Outside, ...)
- $G_0 = 1 + 1 * 0.9 + 1 * 0.9^2 + \cdots = 10.0$

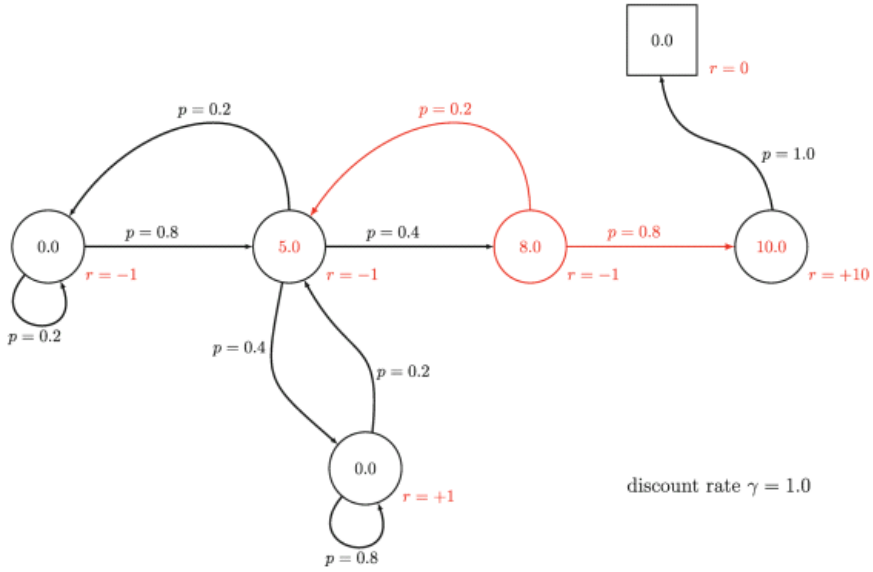
i. **Initial Values**:
   - Initial values for state Room 2 and state Found item are randomly chosen as 5.0 and 10.0, respectively.
   - These initial values serve as starting points for estimating the expected return for other states in the MRP.

ii. **Computing Expected Return for State Room 3**:
   - Given the transition probabilities and values of successor states (Found item and Room 2), the expected return for state Room 3 is computed using the Bellman equation.
   - The Bellman equation considers the immediate reward, transition probabilities, and values of successor states to estimate the value of the current state (Room 3).

iii. **Accuracy of Estimated Value**:
   - The estimated value for state Room 3 may not be accurate initially due to randomly assigned values and the limited scope of the computation.
   - However, by including all states in the state space and iterating the process multiple times, the estimated values will converge towards the true values.

iv. **Significance of Estimating State Values**:
   - Estimating state values is crucial as it helps the agent make informed decisions by identifying states with higher expected returns.
   - By selecting actions that lead to high-value states, the agent can maximize its long-term expected return and improve its decision-making process.

v. **Dynamic Programming and Value Computation**:
   - The true values of states in the MRP are computed using dynamic programming and the Bellman equation (Eq. 2.7), demonstrating the iterative nature of value computation.
   - Dynamic programming is highlighted as an effective method for solving Markov Decision Processes (MDPs) by iteratively updating state values based on rewards and transition probabilities.

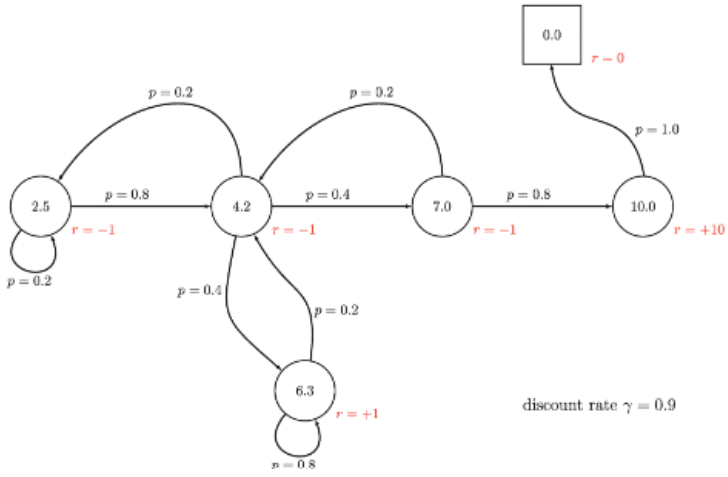


5. Markov Decision Process (MDP)
   The MDP is described as containing all the necessary components for decision-making, including states, rewards, actions, and policies. It is defined as a tuple with the following components:
   - **State Space (S)**: A finite set of states representing possible configurations or observations of the environment.
   - **Action Space (A)**: A finite set of actions that the agent can take in the environment.
   - **Dynamics Function (P)**: Also known as the transition model, this function specifies the probability of transitioning to a successor state when the agent is in a current state and takes a specific action.
   - **Reward Function (R)**: This function defines the reward signal provided by the environment when the agent is in a particular state and takes a specific action.

   This definition highlights the key elements of an MDP and emphasizes its role in modelling sequential decision-making problems under uncertainty in reinforcement learning scenarios.

$$P(s'|s,a) = P\left[S_{t-1} = s' \,\middle|\, S_t = s, A_t = a\right]$$

$$R(s,a) = \mathbb{E}\left[R_t \,\middle|\, S_t = s, A_t = a\right]$$



- **Dynamics and Rewards**: Functions now depend on agent actions.
- **Merged States**: Found item and End combined into one terminal state.
- **Action Representation**: Lines show valid actions, not transition probabilities.
- **Action-Dependent Rewards**: Rewards vary based on agent actions.
- **Deterministic Environment**: Same action in same state yields consistent outcomes.
- **Example**: Choosing Go outside in Room 2 always leads to Outside state.

We can list all the elements in the set of for our service dog MDP; notice that not all actions are available (or legal) in each state:
S= {Room 1, Room 2, Room 3, Outside, Found item}
A= {Go to room1, Go to room2, Go to room3, Go outside, Go inside, Search}
R = {-1 , -2 , +1, 0, +10}

i. Deterministic Environment:
   1) The service dog MDP operates in a deterministic (stationary) reinforcement learning environment.
   2) Transitioning from the current state to its successor state is based on the current state and the action chosen by the agent.

ii. Transition Probabilities:
   1) In a deterministic environment, legal actions have a transition probability of 1.0, ensuring a definite outcome.
   2) Illegal actions, actions not permitted in the environment, have a transition probability of 0.0.
   3) It is crucial for agents to avoid choosing illegal actions due to enforced checks in most environments.

$$\mathcal{P} = \begin{array}{c} \\ \text{Go to room1} \\ \text{Go to room2} \\ \text{Go to room3} \\ \text{Go outside} \\ \text{Go inside} \\ \text{Search} \end{array} \begin{array}{c} \begin{array}{ccccc} \text{Room 1} & \text{Room 2} & \text{Room 3} & \text{Outside} & \text{Found item} \end{array} \\ \begin{pmatrix} 1.0 & 0 & 0 & 0 & 0 \\ 0 & 1.0 & 0 & 0 & 0 \\ 0 & 0 & 1.0 & 0 & 0 \\ 0 & 0 & 0 & 1.0 & 0 \\ 0 & 1.0 & 0 & 0 & 0 \\ 0 & 1.0 & 0 & 0 & 0 \end{pmatrix} \end{array}$$

$$\mathcal{P} = \begin{array}{c} \\ \text{Go to room1} \\ \text{Go to room2} \\ \text{Go to room3} \\ \text{Go outside} \\ \text{Go inside} \\ \text{Search} \end{array} \begin{array}{c} \begin{array}{ccccc} \text{Room 1} & \text{Room 2} & \text{Room 3} & \text{Outside} & \text{Found item} \end{array} \\ \begin{pmatrix} 0.6 & 0 & 0 & 0.4 & 0 \\ 0 & 1.0 & 0 & 0 & 0 \\ 0 & 0 & 0.2 & 0.8 & 0 \\ 0 & 0 & 0 & 1.0 & 0 \\ 0 & 1.0 & 0 & 0 & 0 \\ 0 & 1.0 & 0 & 0.0 & 0 \end{pmatrix} \end{array}$$

iii. Matrix Representation :
   1) Despite the need for a 3D matrix to represent the dynamics function for the service dog MDP, a 2D plane is used for clarity.
   2) In this representation, each row in the plane corresponds to an action, and each column represents a successor state.

iv. Handling Illegal Actions:
   1) To maintain consistency in the environment, the transition probability to the successor state Room 2 is set to 1.0 for all illegal actions such as Go to room2, Go inside, and Search.
   2) This approach ensures that illegal actions do not impact the state of the environment.

a. Policy
A policy in reinforcement learning is a formal function that assigns probabilities to different actions in a given state. This policy can take two forms: deterministic or stochastic.

i. Deterministic Policy:
   1) In a deterministic policy, the same action is consistently chosen for a specific state.
   2) For instance, if a state offers four possible actions, and action is the most rewarding while action is the least rewarding, a deterministic policy will always select action for that state.

ii. Stochastic Policy:
   1) In contrast, a stochastic policy involves selecting actions based on a probability distribution.

2) Using the same example, a stochastic policy might allocate probabilities as follows: 60% for action a_1 , 20% for action a_2 , 15% for action a_3 , and 5% for action a_4 .
3) This stochastic nature allows for variability in action selection, introducing randomness into the decision-making process based on the assigned probabilities.

iii. Policy in Reinforcement Learning:
- A policy in reinforcement learning is a function that maps each state to a probability distribution over possible actions in that state.
- Policies can be deterministic, where a single action is always chosen, or stochastic, where actions are selected based on probabilities.

iv. Mathematical Representation:
- The policy function is denoted as π, where π(a|s) represents the probability of taking action a in states according to the policy.

$$\sum_{a \in A} \pi(a|s) = 1.0$$

- The policy π can be deterministic, with π(s) returning a specific action, or stochastic, with π(a|s) providing a distribution over actions.
- Number of deterministic probability:
  The formula for calculating the number of possible deterministic policies for a finite Markov Decision Process (MDP) is given by:

  $ \text{Number of Deterministic Policies} = |\mathcal{A}|^{|\mathcal{S}|} $

  where:
  - $ |\mathcal{A}| $ is the size of the action space (number of possible actions).
  - $ |\mathcal{S}| $ is the size of the state space (number of possible states).

  This formula represents the total number of unique deterministic policies that can be defined for a finite MDP based on the available actions and states.

v. Policy Evaluation:
- To evaluate a policy, the expected cumulative reward from a specific states following policy π can be computed using the formula:

$$\pi(a|s) = P\left[A_t = a \,\middle|\, S_t = s\right], \quad \text{for all } s \in \mathcal{S}, a \in \mathcal{A}$$

- This formula calculates the expected value of the cumulative reward starting from state s and discounting future rewards by γ.

b. Value Function for MDP
Value functions in MDPs serve as essential tools for evaluating and comparing different policies based on their expected returns. There are two primary types of value functions in MDPs:

i. State Value Function:
1) The state value function in an MDP estimates the expected return starting from a specific state and following a policy thereafter.

$$V_\pi(s) = \mathbb{E}_\pi\left[G_t \,\middle|\, S_t = s\right], \quad \text{for all } s \in \mathcal{S}$$

2) It quantifies how good or bad it is for the agent to be in a particular state under a given policy.
3) The state value function is denoted as $V_\pi(s)$, where the state and π^s the policy.

ii. State-Action Value Function:
1) The state-action value function in an MDP predicts the expected return from taking a specific action in a given state and then following a policy thereafter.
2) It assists in evaluating the quality of actions in different states under a particular policy.
3) The state-action value function is represented as $Q_\pi(s,a)$, where sis the state, a is the action, and π is the policy.

These value functions play a crucial role in reinforcement learning by guiding the agent towards making decisions that lead to maximizing cumulative rewards over time. By utilizing value functions, agents can learn and adapt their policies to achieve optimal performance in various environments and tasks within the MDP framework.

$$V_\pi(s) = \sum_{a \in A} \pi(a|s) Q_\pi(s,a)$$

iii. Bellman Equation

The Bellman equation for the state value function $V\pi(s)$under a policy π in a Markov Decision Process (MDP) relates the value of a state to the immediate reward obtained in that state and the discounted value of the successor states reached by following the policy. Similarly, the Bellman equation for the state-action value function $Q_\pi(s,a)$under a policy πconnects the value of taking a specific action in a state to the immediate reward received and the discounted value of the successor states reached by following the policy.

Specifically, the Bellman equation for the state value function $V_\pi(s)$under policy $\pi$is given by:

$V_\pi(s)=\sum_a \pi(a|s)\sum_{s',r} p(s',r|s,a)[r+\gamma V_\pi(s')]$

where:

- s is the current state,
- a is the action taken in state s,
- s' is the successor state,
- r is the immediate reward received,
- $\pi(a|s)$is the probability of taking action ain state sunder policy π,
- $p(s',r|s,a)$is the transition probability to reach state s' and receive reward rby taking action ain state s,
- γ is the discount factor,
- $V_\pi(s')$ is the value of the successor state s' under policy π.

Similarly, the Bellman equation for the state-action value function $Q\pi(s,a)$under policy π can be expressed as:

$Q_\pi(s,a)=\sum_{s',r} p(s',r|s,a)[r+\gamma\sum_{a'\pi}(a'|s')Q_\pi(s',a')]$

where:

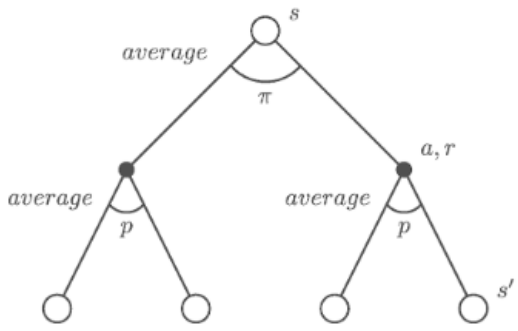
- $Q_\pi(s,a)$is the value of taking action ain state sunder policy π,
- The terms have similar meanings as in the state value function Bellman equation.

These Bellman equations play a fundamental role in reinforcement learning algorithms, as they provide a recursive relationship for updating the value functions based on the rewards and values of successor states, enabling the agent to learn and improve its decision-making process over time under a given policy.

c. Bellman Equation Backup diagram for V_π and Q_π

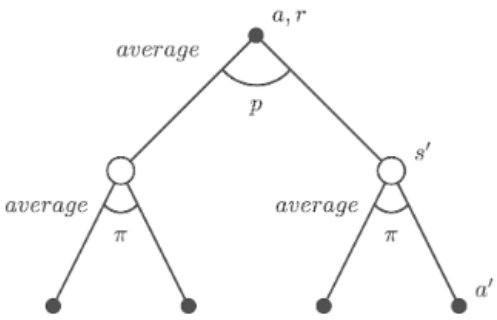
A backup diagram for $V\pi$and $Q\pi$provides a visual representation of how the Bellman equations for the state value function and state-action value function under a policy $\pi$are applied to update the values of states and state-action pairs in a Markov Decision Process (MDP). These diagrams help in understanding the flow of information and value updates during the reinforcement learning process.

1. **Backup Diagram for $V\pi$**:
   - In the backup diagram for V_π, each node represents a state in the MDP.



   - The root node corresponds to the current state s for which we are computing the value under policy π.
   - From the root node, there are edges representing possible actions that can be taken in state s according to policy π.
   - Each action edge leads to successor states that can be reached from state s by taking the corresponding action.
   - The Bellman equation for V_π is applied at each successor state to update the value based on the immediate reward and the discounted value of the next state under policy π.
   - The diagram illustrates how the values of states are recursively updated by propagating information from successor states back to the current state, following the Bellman equation.

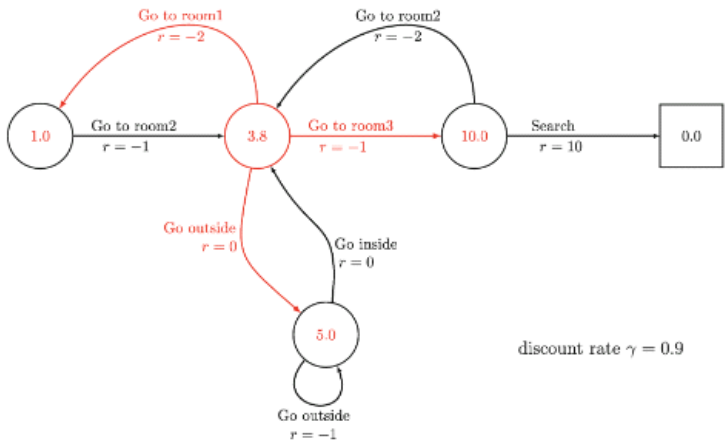
2. **Backup Diagram for Qπ**:
   - The backup diagram for Qπextends the concept to state-action pairs, representing the value of taking a specific action in a given state under policy π.



   - Similar to the V_π diagram, each node in the Qπdiagram corresponds to a state-action pair.
   - The edges from each state-action pair node lead to successor states that can be reached by taking the corresponding action in the current state.
   - The Bellman equation for Q_π is applied at each successor state to update the value of the state-action pair based on the immediate reward and the discounted value of the next state-action pair under policy π.
   - The diagram visually demonstrates how the values of state-action pairs are iteratively adjusted by considering the rewards and values of successor state-action pairs, following the Bellman equation.

In summary, backup diagrams for Vπand Qπprovide a structured visualization of the value update process in MDPs, showcasing how information flows through states and state-action pairs to c ompute the expected returns under a given policy.

d. **Worked Example for V_π using a Service Dog MDP:**
   1. **Scenario Description**:
      - Consider a service dog navigating through different rooms in a building.
      - The dog can be in various states representing different rooms.
      - Actions available to the dog include moving to adjacent rooms or going outside.



$$
\begin{aligned}
V_\pi(Room\ 2) &= 0.33 * (-2 + 0.9 * 1.0) + 0.33 * (-1 + 0.9 * 10.0) \\
&\quad + 0.33 * (0 + 0.9 * 5.0) \\
&= 0.33 * -1.1 + 0.33 * 8 + 0.33 * 4.5 \\
&= 3.76
\end{aligned}
$$

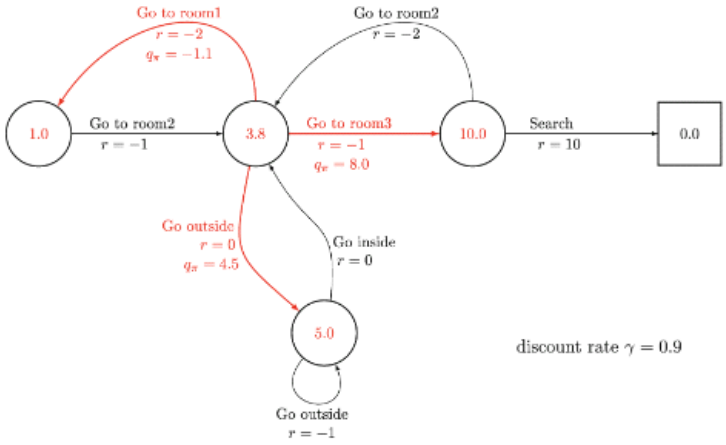
   2. **Policy Selection**:
      Assume a random policy π where the dog's actions are evenly distributed across all legal actions in each state.
   3. **State of Interest**:
      Focus on computing the value for a specific state, let's say "Room 2".
   4. **Actions in State "Room 2"**:
      In "Room 2", the dog has three legal actions: "Go to Room 1", "Go to Room 3", and "Go outside".

5. **Initial Guesses**:
   - Assign initial guessed values for each of the successor states reachable from "Room 2".
6. **Computing $V_\pi(s)$for "Room 2"**:
   - Apply the Bellman equation for Vπto update the value of "Room 2" based on the rewards and values of successor states under policy π.
   - Iterate through the actions available in "Room 2" and calculate the expected return for each action by considering the immediate rewards and the values of successor states.
   - Update the value of "Room 2" by aggregating the expected returns from all actions weighted by the policy probabilities.
7. **Iteration and Convergence**:
   - Repeat the value computation process iteratively until the values stabilize or converge, indicating the estimated value of "Room 2" under policy π.

e. **Worked Example for Q_π using a Service Dog MDP:**

1. **Scenario Description**:
   - Consider a service dog navigating through different rooms in a building.
   - The dog can be in various states representing different rooms.
   - Actions available to the dog include moving to adjacent rooms or going outside.

$$
\begin{aligned}
Q_\pi(Room\ 2,\ Go\ to\ room1) &= -2 + 0.9 * 1.0 \\
&= -1.1 \\
Q_\pi(Room\ 2,\ Go\ to\ room3) &= -1 + 0.9 * 10.0 \\
&= 8.0 \\
Q_\pi(Room\ 2,\ Go\ outside) &= 0 + 0.9 * 5.0 \\
&= 4.5
\end{aligned}
$$



2. **Policy Selection**:
   Assume a random policy πwhere the dog's actions are evenly distributed across all legal actions in each state.
3. **State-Action Pair of Interest**:
   Focus on computing the value for a specific state-action pair, for example, "Room 1" and the action "Go to Room 2".
4. **Initial Guesses**:
   Assign initial guessed values for the state-action pair "Room 1" and "Go to Room 2".
5. **Computing Qπ(s,a)for "Room 1" and "Go to Room 2"**:
   - Apply the Bellman equation for Q_π to update the value of the state-action pair based on the rewards and values of successor state-action pairs under policy π.
   - Consider the immediate reward for taking action "Go to Room 2" from "Room 1" and the value of the successor state-action pair.
   - Update the value of the state-action pair "Room 1" and "Go to Room 2" by incorporating the immediate reward and the discounted value of the successor state-action pair.
6. **Iteration and Convergence**:
   Repeat the value computation process iteratively until the values stabilize or converge, indicating the estimated value of taking action "Go to Room 2" in "Room 1" under policy π. All the agent needs to do is select the action that has the highest state-action value for all states in the state space. And it doesn't need to do a one-step look-ahead anymore

f. Alternative for Bellman Equation Function

In reinforcement learning, the Bellman equations are fundamental for understanding how values of states and state-action pairs are updated based on rewards and transitions in the environment.

Alternative Bellman equations for value functions provide variations to accommodate different scenarios where rewards may be conditioned on successor states. Let's explore the alternative Bellman equations along with the formulas and explanations of each symbol:

Alternative Bellman Equations for Value Functions:

1. **Bellman Equation for Vπwith Reward Conditioned on Successor State**:
   - **Formula**: $V_\pi(s)=\sum_a \pi(a|s)\sum_{s',r} p(s',r|s,a)[r+\gamma V_\pi(s')]$
   - **Explanation**:
     - Vπ(s): Value of state sunder policy π.
     - π(a|s): Probability of taking action ain state sunder policy π.
     - p(s′,r|s,a): Transition probability to successor state s′with reward rafter taking action ain state s.
     - r: Immediate reward received after taking action ain state s.
     - γ: Discount factor for future rewards.
     - $V_\pi(s')$: Value of successor state s′ under policy π.
2. **Bellman Equation for Qπwith Reward Conditioned on Successor State**:
   - **Formula**: $Q_\pi(s,a)=\sum_{s',r} p(s',r|s,a)[r+\gamma\sum a'_\pi(a'|s')Q_\pi(s',a')]$
   - **Explanation**:
     - $Q_\pi(s,a)$: Value of taking action ain state sunder policy π.
     - π(a′|s′): Probability of taking action a′ in successor state s′under policy π.
     - Other symbols have similar meanings as in the Vπequation.
3. **Significance**:
   - These alternative Bellman equations allow for more flexibility in modeling environments where rewards depend on successor states.
   - They enable reinforcement learning algorithms to handle scenarios where immediate rewards are not solely based on the current state-action pair.

By incorporating these alternative Bellman equations into the reinforcement learning framework, agents can effectively learn and make decisions in environments with complex reward structures that are conditioned on successor states.

6. Optimal Policy and Optimal Value

In the context of Markov Decision Processes (MDPs), understanding the concepts of an optimal policy and an optimal value function is crucial for designing effective reinforcement learning algorithms. Let's delve into the explanations, formulas, and symbols associated with the optimal policy and optimal value function in MDPs:

Optimal Policy in MDP:

1. **Definition**:
   - An optimal policy in an MDP is a policy that maximizes the expected cumulative rewards over time, leading to the highest possible return for the agent.
2. **Formula**:
   - The optimal policy π∗is defined as: π∗=arg max π $V_\pi(s)$where $V_\pi(s)$represents the value function under policy π, and argmax denotes the policy that maximizes the value function.
3. **Symbols**:
   - π∗: Optimal policy.
   - Vπ(s): Value function for policy π.
   - s: State in the MDP.

Optimal Value Function in MDP:

1. **Definition**:
   - The optimal value function in an MDP represents the maximum expected return achievable by following the optimal policy.
2. **Formula**:
   - The optimal state value function V∗(s)is defined as: V∗(s)=maxπVπ(s)where V∗(s)denotes the optimal value of state sacross all policies.
3. **Symbols**:
   - V∗(s): Optimal value function for state s.
   - maxπ: Maximum over all policies.
   - Other symbols used are consistent with those in the optimal policy explanation.

Significance:

- The optimal policy guides the agent to make decisions that lead to the highest rewards in each state.
- The optimal value function quantifies the maximum expected return achievable by following the optimal policy in each state.